

Student Meal Planner

Component Hierarchy

A Design Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

How the Frontend is Organized

Every page in Grubs4Scrubs follows the same pattern. One file per page, multiple function components inside that file, and a default export that wraps everything with the Navbar and Footer. Internal components handle sections of the page. Data flows down through props. No component libraries, no state management libraries, just React with `useState` and `useNavigate`.

The tree below shows which components live where and how they nest inside each other.

App Level (main.jsx)

The root of the app is `main.jsx`, which sets up React Router. Every route points to a page component.

```
App
├── Route: /                → HomePage
├── Route: /dashboard       → DashboardPage
├── Route: /recipes         → RecipesPage
├── Route: /recipes/:id     → RecipeViewPage
├── Route: /mealplanner     → MealPlannerPage
├── Route: /shoppinglist    → ShoppingListPage
├── Route: /login           → LoginPage
└── Route: /signup          → SignUpPage
```

Shared Components (src/Components/)

These get imported by multiple pages.

```
Navbar      – top navigation bar, appears on every page
Footer      – bottom footer, appears on every page
```

Both are imported in each page's wrapper function. They're not in a layout component because each page controls its own structure.

HomePage (src/Pages/HomePage.jsx)

```
HomePage (default export – wraps with Navbar + Footer)
├── HomeHero      – hero banner with CTA button
├── HomeFeatures  – feature cards (3 cards: meal planner, recipes,
budget)
```

- |— HomeFeatured — featured recipe cards (clickable, navigate to / recipes/:id)
- |— HomeHowItWorks — step-by-step explanation section
- |— HomeCreateAccountTab — sign-up CTA section at the bottom

CSS: HomePage.css

DashboardPage (src/Pages/DashboardPage.jsx)

DashboardPage (default export — wraps with Navbar + Footer)

- |— DashboardHeader — greeting and date display
- |— DashboardMetrics — stat cards (budget, saved recipes, planned meals, shopping items)
- |— DashboardMealPlan — today's meals broken into breakfast/lunch/dinner
 - | — DashboardDayColumn — single meal type column with icon and meal info
- |— DashboardFeatured — recommended recipe cards
- |— DashboardShoppingList — mini shopping list with checkable items (uses useState)

CSS: DashboardPage.css

The shopping list uses `useState` for toggling checkboxes. Each item tracks its own checked state.

RecipesPage (src/Pages/RecipesPage.jsx)

RecipesPage (default export — wraps with Navbar + Footer)

- |— Recipe filter buttons — category pills (All, Breakfast, Lunch, Dinner, Snack)
- |— Recipe cards grid — grid of recipe cards
 - | — Individual card — image, title, tags, prep time, budget (clickable → /recipes/:id)
 - | — Add recipe card — dashed border card for adding new recipes

CSS: RecipesPage.css

Uses `useNavigate` to send users to the recipe detail page when they click a card.

RecipeViewPage (src/Pages/RecipeViewPage.jsx)

RecipeViewPage (default export — wraps with Navbar + Footer)

- |— RecipeViewHero — full-width image (or emoji fallback), gradient overlay, meta bar
 - | — Edit button — pencil icon, opens RecipeEditModal

- | └─ Delete button – trash icon, DELETE request + redirect to / recipes
- |─ RecipeViewIngredients – checkable ingredient list (uses useState)
- |─ RecipeViewNutrition – nutrition info grid (calories, protein, carbs, fat)
- |─ RecipeViewInstructions – numbered step-by-step instructions
- |─ RecipeViewTip – tip card with orange glow styling
- └─ RecipeEditModal – modal form pre-filled with recipe data, sends PUT on save

CSS: RecipeViewPage.css (plus RecipesPage.css for shared modal styles)

Uses `useParams` to grab the recipe ID from the URL. Fetches recipe data from the backend API. The edit modal parses JSON-stored ingredients and instructions back into readable text for editing, then converts them back to JSON arrays on save.

MealPlannerPage (src/Pages/MealPlannerPage.jsx)

```
MealPlannerPage (default export – wraps with Navbar + Footer)
├─ MealPlannerHeader        – title and description
├─ MealPlannerDayTabs       – day selector pills (Mon through Sun)
├─ MealPlannerGrid          – 3-column grid for selected day
|   └─ MealPlannerSlot      – individual meal slot (filled or empty with
dashed border)
└─ MealPlannerWeekOverview – weekly stats (total meals planned, total
estimated cost)
```

CSS: MealPlannerPage.css

Uses `useState` for the selected day. Week overview calculates totals with a for loop over the meal data object.

ShoppingListPage (src/Pages/ShoppingListPage.jsx)

```
ShoppingListPage (default export – wraps with Navbar + Footer)
├─ ShoppingListHeader       – title, item count, total cost
├─ Add item form            – inputs for name, quantity, price with add button
├─ ShoppingListItem (×N)    – one per item, two modes:
|   └─ Normal mode          – checkbox, name, quantity, price, edit button,
delete button
|   └─ Edit mode            – inline input fields for name/qty/price, save
(check) and cancel (X) buttons
└─ Empty state              – message when no items exist
```

CSS: ShoppingListPage.css

Fully connected to the backend API. Uses `editingId` state to track which item is being inline-edited. Only one item can be in edit mode at a time. The edit row gets a blue-tinted background to distinguish it visually.

LoginPage (src/Pages/LoginPage.jsx)

```
LoginPage (default export – wraps with Navbar + Footer)
├── Login form card          – glass card with form fields
│   ├── Email input
│   ├── Password input
│   ├── Submit button
│   └── Google sign-in button (UI only, not functional)
└── Link to SignUpPage
```

CSS: LoginPage.css

UI only. Not connected to the backend auth API yet.

SignUpPage (src/Pages/SignUpPage.jsx)

```
SignUpPage (default export – wraps with Navbar + Footer)
├── Sign-up form card        – glass card with form fields
│   ├── Username input
│   ├── Email input
│   ├── Password input
│   ├── Confirm password input
│   └── Submit button
└── Link to LoginPage
```

CSS: SignUpPage.css

UI only. Not connected to the backend auth API yet.

Data Flow Pattern

Props flow down. No prop drilling beyond one level because each page's internal components are siblings, not deeply nested. The page wrapper function holds the state (if any) and passes it to the section components.

Example from DashboardPage:

```
DashboardPage
└─> passes shoppingItems + toggle function to DashboardShoppingList
```

- passes todayMeals to DashboardMealPlan
- DashboardMealPlan passes individual meal to DashboardDayColumn

CSS Architecture

One CSS file per page. Each file uses `ComponentName-element-subelement` naming. No Tailwind. Shared base styles live in `HomePage.css` and `Components.css`, which every page imports alongside its own stylesheet.

```
src/Pages/
├── HomePage.css           – base styles + home page specific
├── DashboardPage.css      – dashboard specific
├── RecipesPage.css        – recipes grid + add recipe modal specific
├── RecipeViewPage.css     – recipe detail + edit button specific
├── MealPlannerPage.css    – meal planner specific
├── ShoppingListPage.css   – shopping list + inline edit specific
├── LoginPage.css          – login form specific
└── SignUpPage.css         – sign-up form specific

src/Pages/
└── Components.css        – navbar, footer, shared component styles
```